

1. Factors for a Good Agile Team To maximize marks, emphasize the human element of agile development. A successful team relies on:

- Trust:** Members must trust each other's technical competence and commitment to the project.
- Skill Distribution:** The mix of skills must be appropriate for the specific problem; for instance, a team needs both UI experts and system architects if the project requires it.
- Self-Organization:** The team should be autonomous, organizing its own work tasks and schedules rather than following top-down instructions.
- Collaboration (Pair Programming):** Practices like pair programming encourage constant learning and check-and-balance on code quality.

2. Factors Leading to Software Project Failure Project failure is rarely due to a single technical error. Major "bad factors" include:

- Unrealistic Deadlines:** Often established by managers outside the development group, leading to rushed, low-quality work.
- Poorly Defined Scope:** Failure to understand the problem or set realistic expectations at the start.
- Scope Creep:** Changing customer requirements that are not reflected in schedule or resource adjustments.
- Human Difficulties:** Lack of communication among project staff or unforeseen human issues that result in delays.

3. Critical Practices in Software Project Management The Airlie Council identifies these essential practices for project integrity:

- Formal Risk Management:** Proactively identifying and resolving risks before they become problems.
- Empirical Cost/Schedule Estimation:** Using data-driven metrics (like LOC or FP) rather than "gut feelings".
- Metrics-Based Management:** Quantitative tracking of progress.
- Earned Value Tracking:** Assessing the "percent of completeness" through quantitative analysis.
- Defect Tracking:** Monitoring bugs against quality targets.

4. Data Dictionary Model for a Crypto Trading System A data dictionary provides standardized definitions for system entities to ensure consistent interpretation.

- Example Entry 1:** *Name:* **Trade_Order** | *Description:* A request to buy or sell a specific cryptocurrency | *Type:* Entity | *Attributes:* **Order_ID, Asset_Name, Quantity, Price.**
- Example Entry 2:** *Name:* **Asset_Price** | *Description:* The current real-time market value of a digital asset | *Type:* Attribute | *Source:* External Market API.

5. Role of Data Dictionary in Car Parts In complex systems like car parts management, the data dictionary:

- Ensures **Unique Naming:** Every part (e.g., **Cylinder_Head_v2**) has a unique name and description to prevent designer misinterpretation.
- Supports **CASE Tools:** It manages information from all system models in a central repository, allowing for automated documentation generation.

6. Factors Considered When Selecting Project Team Structure According to Pressman, a manager must consider:

- Problem Difficulty:** Highly complex problems may require more formal, hierarchical structures (Closed Paradigm).
- Project Size:** Measured in LOC or FP; larger projects need more structured coordination.
- Modularization Degree:** If a problem is easily compartmentalized, team members can work independently (Synchronous Paradigm).
- Rigidity of Delivery Dates:** Strict deadlines might favor a centralized structure for faster decision-making.

7. Product Scope and Project Scope

- Product Scope:** A narrative description of the **functions, features, data, and behaviors** delivered to the end-user. It identifies *what* is being built.
- Project Scope:** Describes **all the work required** to produce the product, including the process, resources, and administrative tasks.

8. Class Diagram with Aggregation for a Petrol Bunk

- Definition:** Aggregation is a "part-of" relationship where one object is composed of other objects.
- Diagram Directions:** Draw a central box labeled **Petrol Bunk**. Connect it to smaller boxes labeled **Fuel Pump**, **Storage Tank**, and **Payment Terminal**.
- Symbol:** Place a **diamond shape** on the line exactly where it touches the **Petrol Bunk** box.

9. Class Diagram with Generalization for a Petrol Bunk

- Definition:** Generalization represents inheritance, where specialized classes inherit from a general parent.
- Diagram Directions:** Draw a box at the top labeled **Fuel**. Below it, draw boxes for **Petrol** and **Diesel**.
- Symbol:** Draw a solid line from the child boxes (**Petrol**, **Diesel**) ending in a **hollow triangle** pointing at the parent (**Fuel**).

10. Three System Organizational Styles (LIBSYS) Using a library system (LIBSYS) as an example:

- Repository Model:** All data (book records, user IDs) is stored in a central database accessed by sub-systems like **Ordering** or **Cataloguing**.
- Client-Server Model:** The system provides services (e.g., **Search**) via a server, while users interact via a web browser "client".
- Layered Model:** Organizes the system into layers: UI at the top, Information Retrieval in the middle, and Database management at the bottom.

11. Numerical Calculation (Website Development) To score full marks, show every formula and step:

- I. Value Adjustment Factor (VAF):** $0.65 + (0.01 \times 33) = 0.98$.
- II. Function Point Estimate (FP):** $300 \times 0.98 = 294$ FP.
- III. Size in LOC:** $294 \text{ FP} \times 50 \text{ LOC/FP} = 14,700$ LOC.
- IV. Effort Required:** $294 \text{ FP} / 14 \text{ FP/pm} = 21$ person-months.
- V. Project Cost (Total):** $21 \text{ person-months} \times ₹90,000 = ₹1,890,000$. (Note: Using LOC method leads to the same result if productivity is converted: $14 \text{ FP/pm} \times 50 \text{ LOC/FP} = 700 \text{ LOC/pm}$. $14,700 / 700 = 21 \text{ pm}$.)

12. State Machine Model for a Hair Dryer

- Diagram Directions:** Draw rounded rectangles for states: **OFF**, **Cool_Air**, **Low_Heat**, **High_Heat**.
- Transitions:** Label arrows with triggers. An arrow from **OFF** to **Cool_Air** is triggered by **Switch_On**. Further presses of a **Heat_Button** cycle the dryer through heat levels.

13. Object Models Object models represent both system data and its processing as interacting objects.

- Diagram:** Draw a vertically oriented rectangle divided into three sections: 1. **Class Name**, 2. **Attributes** (state variables), 3. **Operations** (services/methods).

14. Sequence Model for Insulin Pump

- Diagram Directions:** Lay out objects **Sensor**, **Controller**, and **Pump** horizontally at the top.
- Interactions:** Draw dashed vertical lifelines down from each. Draw horizontal arrows between them: 1. **Sensor** sends **sugarLevel()** to **Controller**. 2. **Controller** sends **computeDose()** to itself. 3. **Controller** sends **inject()** to **Pump**. **Critical:** Use thin vertical activation boxes on the lifelines.

15. Use Case Model for a Petrol Pump

- Actors:** **Customer** (stick figure).
- Use Cases:** Draw ovals for **Insert_Card**, **Select_Fuel_Type**, **Dispense_Fuel**, and **Pay_Bill**.
- Diagram:** Enclose the ovals in a "system boundary" box and connect the **Customer** to all relevant ovals with solid lines.

16. Multiple Inheritance

- Definition:** A class inherits attributes and operations from more than one parent class.
- Diagram:** Draw a child class **Amphibious_Vehicle**. Above it, draw two parents: **Land_Vehicle** and **Water_Vehicle**. Connect the child to **both** parents using lines with hollow triangles.

17. CASE Workbenches Workbenches are integrated sets of tools that support specific software process activities.

- Example:** A **Testing Workbench** might include a test data generator, execution monitor, file comparator, and report generator.

18. Architectural System Characteristics The chosen architecture affects emergent system properties:

- Performance:** Requires localizing critical operations.
- Security:** Requires a layered structure with critical assets in the middle.
- Safety:** Requires isolating safety-critical components.
- Availability:** Requires redundant components for backup.
- Maintainability:** Requires fine-grain, self-contained components.

19. Packing Robot System Block Diagram

- Diagram Directions:** Based on Sommerville's model, draw sub-system boxes.
- Flow:** **Vision System** sends data to **Object Identification**. This identifies items and signals the **Arm Controller** and **Gripper Controller** to move them. A **Conveyor Controller** manages the belt.

20. Architectural Decisions Architects must answer fundamental questions:

- Is there a generic architecture template to use?
- How will the system be distributed across processors?
- What architectural styles (e.g., Layered, Client-Server) are appropriate?
- How will sub-systems be decomposed into modules?

21. Film and Picture Library (Client-Server) In this model, a central **Server** stores high-volume media files. Multiple **Clients** (users) request specific pictures or films over a network. The server processes the request and streams the data back to the client's local machine for display.

22. Control Styles and Types Control models coordinate sub-system execution:

- Centralized Control:** One sub-system acts as a manager.
 - Call-Return Model:* Top-down subroutine calls (sequential).
 - Manager Model:* One process controls starts/stops of others (concurrent).
- Event-Based Control:** Driven by external stimuli.
 - Broadcast:* An event is sent to all sub-systems.
 - Interrupt-Driven:* Hardware signals force immediate handling (real-time).

23. OSI Model and Reference Architectures Reference architectures, like the 7-layer **OSI Model**, provide a standardized "map" for complex systems. It ensures different vendor products can communicate by following layers (Physical, Data Link, Network, Transport, Session, Presentation, Application).

24. Characteristics of Object-Oriented Design (OOD)

- Encapsulation:** Objects hide their internal state and representation.
- Loose Coupling:** Objects are independent and only interact via interfaces.
- Message Passing:** Communication is done by calling methods/services of other objects.
- Real-world Mapping:** Objects directly represent system or real-world entities.

25. Transponder Object (Java Code) As an **Active Object**, it runs its own thread with an infinite loop:

```
class Transponder extends Thread {
    public void run() {
        while (true) {
            // update position autonomously
        }
    }
}
```

26. Weather Mapping System Subsystem Diagram

- Diagram Directions:** Use UML Package symbols (folder shapes) labeled with the «**subsystem**» stereotype.

- Subsystems:** Group objects into **Interface**, **Data Collection**, **Data Processing**, and **Instruments**.

27. Weather Station Use Case Diagram

- Actors:** **Weather Station** and **Weather Data Collection System**.

- Use Cases:** **Startup**, **Shutdown**, **Report**, **Calibrate**, **Test**.

- Diagram:** Stick figure actors connected to use case ovals inside a boundary box.

28. Approaches to Object Identification There are four common ways to find system objects:

- 1.**Grammatical Analysis:** Using nouns as objects and verbs as operations.

- 2.**Tangible Entities:** Identifying physical things (e.g., a thermometer).

- 3.**Behavioral Approach:** Identifying participants in system behaviors.

- 4.**Scenario-Based:** Finding objects required to complete a use-case scenario.

29. Design Evolution by Adding Pollution Monitoring OOD simplifies change through **Information Hiding**. To add pollution monitoring, you simply create a new **Air quality** object class and pollution sensors (**NOmeter**, etc.). You add a **reportAirQuality()** method to the main interface, leaving existing weather sensors completely **unaffected**.

30. Organizational Paradigms Suggested by Constantine

- Closed Paradigm:** Traditional hierarchy; good for routine, repetitive work.

- Random Paradigm:** Loose structure; relies on individual initiative for breakthroughs.

- Open Paradigm:** Collaborative problem solving with formal controls.

- Synchronous Paradigm:** Compartmentalized work with minimal active communication.

31. Types of Estimation Techniques Managers should reconcile at least two techniques:

- LOC-Based:** Uses historical lines of code data.

- FP-Based:** Uses functional complexity metrics.

- Use-Case Based:** Translates use-case counts into effort.

- Empirical Models:** Statistical formulas derived from past project data.

32. COCOMO Model The **COCOMO II** hierarchy includes sub-models for different stages:

- 1.**Application Composition:** Used during early prototyping.

- 2.**Early Design Stage:** Used once requirements are stable.

- 3.**Post-Architecture Stage:** Used during the construction phase.

33. Stakeholders Involved in Software Project Management They are categorized into five constituencies:

- 1.**Senior Managers:** Define high-level business issues.

- 2.**Project (Technical) Managers:** Plan and control practitioners.

- 3.**Practitioners:** Deliver the technical skills (coders, testers).

- 4.**Customers:** Specify the system requirements.

- 5.**End-Users:** Interact with the final software product